Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

- Clase de probleme (6) -

O ierarhie spațiu - timp a problemelor

Convenții

- din punct de vedere al timpului de execuție o operație elementară are complexitatea $O(1)$;
- complexitate spațială: $d \leq n$ se stochează folosind $O(\lg n)$ biți;
- spațiul ocupat de datele de intrare nu participă la calculul complexității spațiale a algoritmului.

Clase de probleme rezolvabile determinist

Fie $Q : I \to \{0,1\}^*$ o problemă de decizie rezolvabilă prin mulțimea algoritmilor deterministi Alg_Q și $f : \mathbb{N}^* \to \mathbb{R}_+^*$ o funcție (numită funcție de limitare). Notăm:

- $n = dim(i)$ este dimensiunea datelor i;
- $timp(Alg, i)$ este timpul consumat de calculul $Alg(i)$;
- $spatiu(Alg, i)$ este spațiul consumat de calculul $Alg(i)$.

Definim următoarele mulțimi:

$$Alg_Q^T(f) = \{ Alg \in Alg_Q \mid \forall i \in I \text{ avem } timp(Alg, i) = O(f(n)) \}$$

$$Alg_Q^S(f) = \{ Alg \in Alg_Q \mid \forall i \in I \text{ avem } spatiu(Alg, i) = O(f(n)) \}$$

Def:

Fie $f : \mathbb{N}^* \to \mathbb{R}_+^*$ o funcție. Definim clasele de probleme rezolvabile prin algoritmi deterministi cu limitare f temporală, respectiv spațială astfel:

$$TIME(f) = \{ Q \mid Alg_Q^T(f) \neq \emptyset \}$$

$$SPACE(f) = \{ Q \mid Alg_Q^S(f) \neq \emptyset \}$$

Cazuri particulare:

$$P = PTIME = \bigcup_{p \text{ polinom}} TIME(p)$$

$$PSPACE = \bigcup_{p \text{ polinom}} SPACE(p)$$

$$LOGSPACE = \bigcup_{k \geq 1} SPACE(k \cdot \lg n)$$

unde p este un polinom cu valori pozitive.

Def:

S.n. GAP (problema accesibilității unui graf)

problema:

"Fiind date un graf neorientat $G = (V, E)$ și două noduri $i, j \in V$ să se stabilească dacă există drum în G între i și j".

Obs:

Problema GAP este o problemă de decizie.

Prop 1 :

$GAP \in SPACE(\lg^2 n)$.

Dem

Construim un algoritm determinist care să rezolve problema GAP într-un spațiu $O(\lg^2 n)$.

```
GAP_S (G, i, j) // G = (V, E), n = card(V)
    for h = 0 .. n-1
        if drum(i, j, h) return 1
    return 0
```

```
drum (i, j, lung)
    if lung = 0
    |  return i = j
    if lung = 1
    |  return (i, j) ∈ E
    ℓ = lung / 2
    for k = 1.. n
    |  if drum (i, k, ℓ) si drum (k, j, lung - ℓ)
    |  |  return 1
    return 0
```

- Lungimea maximă a lanțului de apeluri recursive este $\lg n$. Fiecare apel consumă spațiu pentru 5 variabile (i, j, lung, k, e).
- Deoarece se lucrează cu numere $\leq n$, spațiul ocupat la fiecare apel este $O(\lg n)$.
- În total funcția <u>drum</u> consumă $O(\lg^2 n)$ biți.

$\Rightarrow$ <u>complexitatea spațială</u> pentru funcția <u>drum</u> este $O(\lg^2 n)$ biți.

- În GAP_S datele G, i, j nu contribuie la complexitatea spațială, conform convenției făcute.
- Se adaugă spațiul ocupat de variabilele n și h adică $O(\lg n)$ biți.
- Complexitatea spațială a algoritmului GAP_S este $O(\lg n + \lg^2 n) = O(\lg^2 n)$.

$\Rightarrow GAP \in SPACE(\lg^2 n)$.

Obs: $GAP \in PTIME$.

Clase de probleme rezolvabile nedeterminist

- Fie M_Alg un algoritm nedeterminist cu n instrucțiuni. S.n. spațiul de stare al algoritmului pentru datele $i \in I$ un graf orientat $G(i) = (V, E, S_0, S_f)$, unde nodurile din mulțimea V reprezintă stări ale algoritmului, iar arcele din mulțimea E tranziții între stări.

- O stare corespunde valorilor curente ale variabilelor algoritmului și instrucțiunii executate de algoritm la un moment dat;
- O tranziție din starea u în starea v depinde de instrucțiunea executată în starea u;

- S_0 = starea inițială
- S_f = starea corespunzătoare terminării cu succes a algoritmului.

Notații:

- căi(N_Alg, i): drumurile simple $s_0 \dots s_f$ din graful G(i);
- stări(d): stările de pe o cale $d \in$ căi(N_Alg, i);
- timp_cale(d): timpul total necesar execuției tuturor prelucrărilor din calea d;
- spațiu_stare(s): spațiul consumat de starea s, excluzând datele nemodificabile ale algoritmului;

- $N_timp(N_Alg, i) =$

  $= \min\{timp_cale(d) \mid d \in căi(N_Alg, i)\}$

- $N_spațiu(N_Alg, i) =$

  $= \min\{spațiu_cale(d) \mid d \in căi(N_Alg, i)\}$

- $spațiu_cale(d) =$

  $= \max\{spațiu_stare(s) \mid s \in stări(d)\}$.

Obs: N_timp și N_spațiu arată cum se calculează complexitatea temporală și spațială pentru un algoritm nedeterminist.

- Fie $Q : I \to \{0,1\}$ o problemă de decizie rezolvabilă prin mulțimea algoritmilor nedeterminiști N_Alg_Q și $f : \mathbb{N}^* \to \mathbb{R}_+^*$ o funcție. Notăm cu $n = dim(i)$ dimensiunea datelor i. Definim următoarele mulțimi:

$$N_Alg_Q^T(f) = \{ Alg \in N_Alg_Q \mid \forall i \in I \text{ avem } N_timp(Alg, i) = O(f(n)) \}$$

$$N_Alg_Q^S(f) = \{ Alg \in N_Alg_Q \mid \forall i \in I \text{ avem } N_spațiu(Alg, i) = O(f(n)) \}.$$

Def:

Fie $f: \mathbb{N}^* \to \mathbb{R}^*_+$ o funcție. Clasele de probleme rezolvabile prin algoritmi nedeterminiști cu limitare f temporală, respectiv spațială sunt:

$$NTIME(f) = \{Q \mid N_Alg^T_Q(f) \neq \phi\}$$

$$NSPACE(f) = \{Q \mid N_Alg^S_Q(f) \neq \phi\}$$

Cazuri particulare:

$$NP = NPTIME = \bigcup_{p \text{ polinom}} NTIME(p)$$

$$NPSPACE = \bigcup_{p \text{ polinom}} NSPACE(p)$$

$$NLOGSPACE = \bigcup_{k \geq 1} NSPACE(k \cdot \lg n)$$

unde p este un polinom cu valori pozitive.

Prop 2:

$GAP \in NLOGSPACE$

Dem

Construim un algoritm nedeterminist N_GAP_S cu complexitate spațială $O(\lg n)$.

```
N_GAP_S(G, i, j) // G = (V, E), n = card(V)
    v = i
    long = 0
    while v ≠ j
        w = choice(V)
        long = long + 1
        if (v, w) ∉ E sau long ≥ n
            fail
        v = w
    success
```

- Fiecare stare a algoritmului conține doar 4 variabile (n, lung, v, w) care contribuie la consumul de spațiu al algoritmului.
- Instrucțiunea choice construiește noi copii ale algoritmului, fiecare copie având câte copie a fiecărei variabile a algoritmului => fiecare copie a algoritmului conține doar 4 variabile care consumă spațiu.

- Numerele cu care lucrăm ca valori ale variabilelor n, lung, v și w sunt numere $\leq n$ => fiecare copie a algoritmului consumă spațiul $O(\lg n)$.
- Prin urmare, algoritmul N_GAP_S are complexitatea spațială $O(\lg n)$ => $GAP \in NLOGSPACE$.

Obs: $GAP \in NPTIME$.

Relații între clasele de probleme

(T1)

$TIME(f) \subseteq NTIME(f)$

$SPACE(f) \subseteq NSPACE(f)$

(C1)

$LOGSPACE \subseteq NLOGSPACE$

$PTIME \subseteq NPTIME$ (sau $P \subseteq NP$)

$PSPACE \subseteq NPSPACE$

(T 2)

Fie f o funcție de limitare a spațiului consumat de o mulțime de algoritmi nedeterminiști astfel încât $f(n) \geq \lg n$, $\forall n \geq 1$ și f să poată fi calculată cu complexitate spațială $O(f(n))$. Atunci:

$\forall Q$ avem $Q \in NSPACE(f) \Rightarrow Q \in TIME(c^f)$, unde c este o constantă dependentă de problemă.

(C 2)

$NLOGSPACE \subseteq PTIME$

(T 3)

$NSPACE(f) \subseteq SPACE(f^2)$,

unde f este o funcție cu proprietățile:

$f(n) \geq \lg n, \forall n \geq 1$ și f poate fi calculată în complexitate spațială $O(f(n))$.

(C 3)

$PSPACE = NPSPACE$

(T 4)

$NPTIME \subseteq PSPACE$

T 5

$$LOGSPACE \subseteq NLOGSPACE \subseteq PTIME \subseteq NPTIME \subseteq PSPACE = NPSPACE$$

Dem

C 1 $\Rightarrow$ $\begin{cases} LOGSPACE \subseteq NLOGSPACE \\ PTIME \subseteq NPTIME \end{cases}$

C 2 $\Rightarrow$ $NLOGSPACE \subseteq PTIME$

C 3 $\Rightarrow$ $PSPACE = NPSPACE$

T 4 $\Rightarrow$ $NPTIME \subseteq PSPACE$.